SPECS-Transformer: A Transformer Architecture for Volatility Forecasting with Signal-Spectrum Dynamics

Alexandro Bizeul

September 23, 2025

Abstract

Modeling volatility in financial markets has been a major challenge for many years, playing an essential role in portfolio risk management and in the pricing of derivative products such as options. This project proposes a methodological, theoretical, and practical approach, beginning with feature engineering applied to financial time series. The study is initially based on the use of a Variational Autoencoder (VAE) which, through probabilistic latent modeling, makes it possible to generate the dynamics and potential trajectories of future volatility. As the study progressed, more complex architectures were introduced to improve the models' sensitivity.

The evaluation was not limited to classic average error metrics. The detection of volatility swings, which can have a significant impact on an asset's performance, and the amplitude of the signals generated, in order to best reflect reality, were the main points of control for assessing a model's effectiveness throughout the study. In both of these areas, the Transformer architecture proved to be the most effective.

Finally, based on this foundation, we refined the loss function and positional encodings, optimizing either the amplitude or the detection of peaks to achieve a final result with a 95.6% reduction in computation time on the same dataset for better performance.

These results demonstrate the potential of complex models and advanced neural networks for volatility forecasting and open up new possibilities in risk management and algorithmic trading.

Table of Contents

Part I — Volatility Forecasting with VAE-LSTM

1. **Preparing the Data:**
   Construction of volatility targets (e.g., rolling standard deviation), input features and sequences, followed by scaling, sequencing and visualization.

2. **Theoretical Foundations of the Variational Autoencoder:**
   Probabilistic modeling, latent spaces, reparametrization, KL divergence, and training objectives, with a discussion of how the VAE is adapted for volatility estimation.

3. **VAE Implementation in PyTorch:**
   Design of the encoder, decoder, and latent sampling, together with a custom loss function combining reconstruction and KL divergence.

4. **Training the Model:**
   Training loop, monitoring of convergence, loss curves, and early stopping, with visualization of the latent space.

5. **Forecasting Volatility at $T+1$:**
   Extraction of predicted volatility from the latent structure, comparison with realized volatility metrics, correlation analysis, error metrics, and regime-based evaluation.

6. **Discussion & Next Steps:**
   Strengths and limitations of the VAE for volatility modeling, potential integration into position sizing or volatility filters, and possible extensions such as CVAE, attention-based VAE, or diffusion models.

Part II — Volatility Forecasting with VAE-GRU

1. **Data Reconfiguration:**
   Definition of new input/output pairs aligned with forecasting volatility directly at $T+1$ or $T+3$.

2. **GRU-based VAE Model:**
   Construction and explanation of the architecture combining a GRU encoder with a regressor decoder.

3. **Training Setup:**
   Training on historical sequences using a KL-regularized MSE loss.

4. **Evaluation:**
   Comparison of predictions with realized volatility using correlation, normalized RMSE, and directional accuracy.

5. **DAVE model: Loss Enhancement**
   Latent space visualization and interpretation of market regimes or transitions.

6. **Fourier Loss Integration in VAE-GRU.**

Part III — Volatility Forecasting with Transformer-MLP

1. **Model Architecture:**
   Design of a Transformer for volatility forecasting, with encoder layers, temporal pooling, and a prediction head (MLP).

2. **FFT-Former:**
   Combination of signal spectrum dynamics (Fourier) and detected peak amplitude.

3. **SPECS-Transformer:**
   Integration of sinusoidal positional encoders.

Introduction

The main goal of this research project is to introduce **volatility forecasting**, a key feature in risk-adjusted sizing and options-related strategies, by forecasting **future volatility at** $T+1$ using a *Variational Autoencoder* (VAE).

In this research project, we aim to explore whether volatility can be meaningfully **anticipated**, rather than inferred from past realized values or implied metrics such as the VIX. We also seek to provide a foundation for **vega-aware strategies**, for instance adjusting exposure ahead of high-volatility periods. Finally, we intend to deepen our understanding of **probabilistic latent modeling** in a financial context.

Dataset & Scope

We will use the **S&P 500 daily log-returns**, scaled and cleaned as in previous projects that will be available soon. Additional features or targets such as **realized volatility** or the **VIX** may be introduced during model construction.

This research project is not focused on short-term profit optimization, but rather on **quantitative research**, model building, and volatility modeling capacity in a financial time series context.

I Part I — Volatility Forecasting with VAE-LSTM

I.1 Preparing the Data

To train a Variational Autoencoder (VAE) to forecast volatility at $T+1$, we need to clearly define the **input features**, namely historical log returns over a fixed window (e.g., 20 days), and the **target variable**, a proxy for realized volatility in the near future.

I.1.1 Defining the Volatility Target

We define realized volatility at time $t+1$ as the standard deviation of log-returns over a short forward-looking horizon:

$$\text{RealizedVol}_{t+1} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(r_{t+i} - \bar{r})^2} \tag{1}$$

where:

- r_t: the daily log-return at time t,
- N: the size of the future window (typically 3),
- $\bar{r}$: the mean return over the window, defined as

$$\bar{r} = \frac{1}{N}\sum_{i=1}^{N} r_{t+i} \tag{2}$$

This target is shifted to time t, so we can learn to forecast it using only information available up to t.

I.1.2 Feature Engineering for Volatility Modeling

In order to enhance the model's ability to forecast volatility, we introduce several additional features derived from the log returns. These features are engineered to capture various aspects of price behavior, such as sudden changes, regime shifts, or amplitude of market fluctuations.

Absolute Return. The absolute return at time t is defined as:

$$\text{AbsRet}_t = |r_t| \tag{3}$$

It acts as a proxy for local volatility and helps the model identify turbulence zones.

Realized Volatility over Different Windows. We compute the backward-looking realized volatility over multiple windows:

$$\text{RealizedVol}_t^{(n)} = \sqrt{\frac{1}{n}\sum_{i=0}^{n-1}(r_{t-i} - \bar{r})^2} \tag{4}$$

We include three versions:

- Short-term: $n = 5$
- Medium-term: $n = 10$
- Long-term: $n = 20$

Historical Return Range. To capture the price movement span over a window:

$$\text{Range}_t = \max(r_{t-n:t}) - \min(r_{t-n:t}) \tag{5}$$

It reflects compression vs expansion patterns in market dynamics.

All these engineered features are concatenated to the original input sequence to enrich the temporal context provided to the VAE.

I.1.3 Scaling

Both the input sequences X_t and the target volatility values are scaled to $[0, 1]$ using MinMax scaling for stable training and to ensure the decoder output stays in a meaningful range. We also visualize the target volatility series to ensure the presence of variability, outliers, and regime changes, which will be crucial for training a generative model such as the VAE.

Code Implementation

Block 1: Data Import and Target Definition

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler

# Volatility target (T+1): realized volatility over next 3 hours
df = pd.read_csv("../data/Cleaned_data.csv", index_col=0)
pd.set_option('display.max_columns', None)

returns = df["SP500_LogRet_Scaled"].dropna()

window_size = 3
realized_vol = returns.rolling(window=window_size).std()
vol_target = realized_vol.shift(-1).dropna()  # this is our y_t+1

# Re-align dataset to keep only overlapping indices
df = df.loc[vol_target.index]
```

Block 2: Feature Engineering and Windowing

```
#Feature Engineering (from returns and context)
df["AbsRet"] = returns.abs()
df["RealizedVol_5"] = returns.rolling(window=5).std()
df["RealizedVol_10"] = returns.rolling(window=10).std()
df["RealizedVol_20"] = returns.rolling(window=20).std()
df["ReturnRange_10"] = returns.rolling(window=10).apply(
    lambda x: x.max() - x.min(), raw=True
```

```
)

# Drop NaNs introduced by rolling
df.dropna(inplace=True)

# Final feature selection for input X_t
feature_cols = [
    "SP500_LogRet_Scaled",
    "AbsRet",
    "RealizedVol_5",
    "RealizedVol_10",
    "RealizedVol_20",
    "ReturnRange_10",
    'volatility_direction',
    "VIX_Close_Scaled"  # contextual implied volatility
]

# Keep only valid features
feature_cols = [col for col in feature_cols if col in df.columns]
features = df[feature_cols]

# Align features with target
common_index = features.index.intersection(vol_target.index)
features = features.loc[common_index]
vol_target = vol_target.loc[common_index]

sequence_length = 20
X_seq, y_seq = [], []

for i in range(sequence_length, len(features)):
    X_seq.append(features.iloc[i-sequence_length:i].values)
    y_seq.append(vol_target.iloc[i])

X_seq = np.array(X_seq)
y_seq = np.array(y_seq).reshape(-1, 1)

print("Input sequences shape:", X_seq.shape)
print("Target volatility shape:", y_seq.shape)

# Scale input and output
x_scaler = MinMaxScaler()
X_scaled = x_scaler.fit_transform(
    X_seq.reshape(-1, X_seq.shape[-1])
).reshape(X_seq.shape)

y_scaler = MinMaxScaler()
y_scaled = y_scaler.fit_transform(y_seq)

print("Scaled input shape:", X_scaled.shape)
print("Scaled target shape:", y_scaled.shape)
```

This block engineers volatility-sensitive features from log-returns (absolute return, realized volatility over 5/10/20 windows, and a 10-step return range), drops NaNs created by rolling operations, and selects the final input set including contextual signals such as the VIX. Features and targets are reindexed on their common timestamps to avoid misalignment. A sliding window of length 20 converts the dataframe into supervised sequences X_t paired with next-step targets y_{t+1}. Finally, both inputs and targets are MinMax-scaled to $[0, 1]$ to stabilize training and keep the decoder's outputs in a meaningful range.

Block 3: Visualization of the Volatility Target

```
# Plot volatility target
start_date = "2024-01-10"   # Change this as needed
end_date = "2024-02-03"

if end_date:
    vol_sample = vol_target.loc[start_date:end_date]
else:
    vol_sample = vol_target.loc[start_date:]
```

```
# Plot
plt.figure(figsize=(12, 4))
plt.plot(vol_sample, label=f"Realized Vol (T+1) from {start_date}")
plt.title("Target Volatility Forecasting on Selected Time Interval")
plt.xlabel("Date")
plt.ylabel("Volatility")
plt.xticks(rotation=45)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

I.2 Theoretical Foundations of the Variational Autoencoder (VAE)

The Variational Autoencoder (VAE) is a generative model that extends the classical autoencoder by adopting a **probabilistic approach** to representation learning. Instead of mapping an input to a fixed code (latent vector), the VAE learns to map it to a **distribution over latent variables**, from which we can sample.

I.2.1 Intuition

In a classical autoencoder, the encoder maps input x to a deterministic latent vector z, and the decoder reconstructs the input as $\hat{x} = f(z)$.

In a *VAE*, the encoder instead maps input x to a **probability distribution over** z. Formally:

$$z \sim q_\phi(z \mid x), \tag{6}$$

where $q_\phi(z \mid x)$ is the *approximate posterior*, parameterized by the neural network weights ϕ.

The decoder then generates reconstructions from the latent variable:

$$\hat{x} \sim p_\theta(x \mid z), \tag{7}$$

where $p_\theta(x \mid z)$ is the *likelihood distribution*, parameterized by θ.
Thus:

- $q_\phi(z \mid x)$ encodes the probability of latent codes z given an observation x (the encoder),
- $p_\theta(x \mid z)$ defines the probability of reconstructing x given z (the decoder),
- The prior on z is usually chosen as a standard multivariate Gaussian:

$$p(z) = \mathcal{N}(0, I).$$

The key idea is to **regularize the latent space** so that it aligns with this prior distribution, enabling smooth sampling, interpolation, and generative capabilities. This probabilistic formulation is directly grounded in **Bayes' theorem**, which links the posterior $p(z \mid x)$ to the likelihood $p(x \mid z)$ and the prior $p(z)$ [1].

I.2.2 Mathematical Formulation

Let $x \in R^d$ be the input (e.g., a sequence of market features), $z \in R^k$ the latent vector, $q_\phi(z \mid x)$ the encoder (approximate posterior), $p_\theta(x \mid z)$ the decoder (likelihood), and $p(z) = \mathcal{N}(0, I)$ the prior over the latent space.

KL Divergence (definition). For two distributions P and Q on the same support $\mathcal{X}$, the Kullback–Leibler divergence is

$$D_{\mathrm{KL}}(P \,\|\, Q) \;=\; \sum_{x \in \mathcal{X}} P(x)\, \log \frac{P(x)}{Q(x)} \quad \text{(discrete case)}, \tag{8}$$

or, in the continuous case,

$$D_{\mathrm{KL}}(P \,\|\, Q) \;=\; \int_{\mathcal{X}} p(x)\, \log \frac{p(x)}{q(x)}\, dx. \tag{9}$$

It satisfies Gibbs' inequality: $D_{\mathrm{KL}}(P\|Q) \geq 0$, with equality iff $P = Q$ a.e.

ELBO (derivation sketch). Starting from the marginal log-likelihood:

$$\log p(x) \;=\; \log \int p_\theta(x \mid z)\, p(z)\, dz. \tag{10}$$

Insert the approximate posterior $q_\phi(z \mid x)$:

$$\log p(x) = \log \int q_\phi(z \mid x)\, \frac{p_\theta(x \mid z)\, p(z)}{q_\phi(z \mid x)}\, dz \tag{11}$$

$$= \log\, E_{q_\phi(z|x)} \left[\frac{p_\theta(x \mid z)\, p(z)}{q_\phi(z \mid x)} \right]. \tag{12}$$

By Jensen's inequality ([2]), which states that $\log E[Y] \geq E[\log Y]$, we obtain

$$\log p(x) \;\geq\; E_{q_\phi(z|x)} \left[\log \frac{p_\theta(x \mid z)\, p(z)}{q_\phi(z \mid x)} \right] \tag{13}$$

$$= \; E_{q_\phi(z|x)} \big[\log p_\theta(x \mid z) \big] \;-\; D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big). \tag{14}$$

This lower bound is the *Evidence Lower Bound (ELBO)*:

$$\log p(x) \;\geq\; \underbrace{E_{q_\phi(z|x)} \big[\log p_\theta(x \mid z) \big]}_{\text{reconstruction likelihood}} \;-\; \underbrace{D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}_{\text{latent regularization}}. \tag{15}$$

Why it matters here. The expectation term encourages accurate reconstruction (capturing fine-grained volatility patterns), while the KL term regularizes the latent space toward $p(z) = \mathcal{N}(0, I)$, preventing memorization and enabling robust sampling of volatility regimes.

I.2.3 Reparameterization Trick

To make $z \sim q_\phi(z \mid x)$ differentiable, we use the *reparameterization trick*:

$$z = \mu(x) + \sigma(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \tag{16}$$

where:

- $\mu(x) \in R^k$ is the vector of means output by the encoder network,
- $\sigma(x) \in R^k$ is the vector of standard deviations (obtained by exponentiating the log-variance predicted by the encoder),
- $\epsilon \sim \mathcal{N}(0, I)$ is an auxiliary noise variable sampled from a standard normal distribution,
- $\odot$ denotes the **element-wise (Hadamard) product**.

Formally, if $a = (a_1, a_2, a_3)^\top$ and $b = (b_1, b_2, b_3)^\top \in R^3$, then

$$a \odot b = \begin{pmatrix} a_1 b_1 \\ a_2 b_2 \\ a_3 b_3 \end{pmatrix}.$$

This formulation ensures that the sampling operation remains differentiable with respect to $\mu(x)$ and $\sigma(x)$, allowing gradients to propagate through the encoder during backpropagation.

I.2.4 Final Loss Function

We use the Euclidean (L2) norm on R^d: for $v = (v_1, \ldots, v_d)$,

$$\|v\|_2 \;=\; \Big(\sum_{i=1}^{d} v_i^2\Big)^{1/2}, \qquad \|v\|_2^2 \;=\; \langle v, v\rangle \;=\; \sum_{i=1}^{d} v_i^2. \tag{17}$$

Hence the reconstruction term $\|x - \hat{x}\|^2$ denotes the squared L2 distance between x and its reconstruction $\hat{x}$.

The training loss is

$$\mathcal{L}_{\text{VAE}}(x) \;=\; \underbrace{\|x - \hat{x}\|_2^2}_{\text{Reconstruction Loss}} \;+\; \beta \cdot \underbrace{D_{KL}(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I))}_{\text{Latent Regularization}}, \tag{18}$$

where the coefficient β controls the trade-off between **fidelity** and **generativity** (the β-VAE formulation).

I.2.5 Application to Volatility Estimation

In our case, x is a **sequence of market features** (log returns, realized volatility, VIX, *etc.*), $\hat{x}$ is the reconstructed sequence, and the true interest lies in the **latent space**: we want z to encode **volatility patterns**. Optionally, we can add a **volatility head** that maps z directly to a prediction of volatility at $T + 1$. Thus, the VAE becomes a tool to compress market context into a **probabilistic latent code**, capture **hidden regimes** or structure in volatility dynamics, and generate forecasts based on a compact, regularized representation.

In the next section, we will implement a `VolatilityVAE` architecture in PyTorch, consisting of an encoder mapping sequences to $(\mu, \log \sigma^2)$, a reparameterized latent sampling layer, a decoder reconstructing the input, and a loss function combining MSE and KL divergence.

I.3 VAE Implementation in PyTorch

In this section, we implement two versions of the Variational Autoencoder (VAE) adapted to our volatility forecasting task. Both models share the same underlying principle: learn a compact latent representation of the market context and leverage it to reconstruct inputs or predict volatility-related patterns. We distinguish two architectures.

I.3.1 VolatilityVAE_Flat

This version treats the input time series sequence as a flat vector. The encoder first flattens the input sequence and passes it through feedforward layers. The latent space is then sampled via the reparameterization trick, and the decoder reconstructs the entire input sequence. The loss combines a mean squared error term between the input and its reconstruction with a KL divergence term between the learned latent distribution and a standard normal prior. This model is effective for **dimensionality reduction** and **clustering** and for visualizing latent volatility regimes.

I.3.2 VolatilityVAE_LSTM

This variant preserves the temporal structure of the input. The encoder is an LSTM that processes each time step and returns a hidden summary, from which the latent space is sampled using the final hidden state. The decoder is another LSTM that reconstructs the input sequence; alternatively, it can predict only the volatility at $T + 1$ directly from the latent vector as an optional head. The loss remains a combination of reconstruction loss and KL divergence. This model is better suited for capturing **temporal dependencies** and **dynamic patterns** in volatility evolution.

I.3.3 Shared Training Data

Both models are trained on the same dataset: `X_scaled`, which contains input sequences of shape $(n_samples, seq_len, n_features)$, and `y_scaled`, which holds the target volatility at $T + 1$ and can optionally be used for supervision. In both cases, we compare the quality of reconstruction, the structure of the learned latent space, and the model's ability to reflect and forecast volatility patterns.

I.3.4 VolatilityVAE_Flat

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class VolatilityVAE_Flat(nn.Module):
    def __init__(self, input_dim, seq_len, latent_dim=10, hidden_dim=64, beta=1.0):
        super().__init__()
        self.input_dim = input_dim
        self.seq_len = seq_len
        self.latent_dim = latent_dim
        self.hidden_dim = hidden_dim
        self.beta = beta

        # Encoder
        self.encoder = nn.Sequential(
            nn.Flatten(),  # (batch, seq_len, input_dim)     (batch, seq_len *
                input_dim)
            nn.Linear(seq_len * input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU()
        )
        self.mu_layer = nn.Linear(hidden_dim, latent_dim)
        self.logvar_layer = nn.Linear(hidden_dim, latent_dim)

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, seq_len * input_dim),
            nn.Sigmoid()  # sigmoid if input data scaled to [0, 1]
        )

    def encode(self, x):
        h = self.encoder(x)
        mu = self.mu_layer(h)
        logvar = self.logvar_layer(h)
        return mu, logvar

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        recon_flat = self.decoder(z)
        return recon_flat.view(-1, self.seq_len, self.input_dim)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        recon = self.decode(z)
        return recon, mu, logvar

    def compute_loss(self, x, recon, mu, logvar):
        recon_loss = F.mse_loss(recon, x, reduction='mean')
        kl_loss = -0.5 * torch.mean(1 + logvar - mu.pow(2) - logvar.exp())
        total_loss = recon_loss + self.beta * kl_loss
        return total_loss, recon_loss, kl_loss
```

The `VolatilityVAE_Flat` class implements a basic feed-forward VAE adapted for volatility sequences. The encoder flattens each input sequence into a vector, applies dense layers with ReLU activations, and outputs the mean μ and log-variance $\log \sigma^2$ of the latent distribution. The reparameterization trick is used to sample z while maintaining differentiability. The decoder then reconstructs the original sequence from z through fully connected layers and a sigmoid activation (since the input data is scaled to $[0, 1]$). The loss function combines the reconstruction error (MSE) and the KL divergence, weighted by the parameter β. This design makes the `FlatVAE` effective for dimensionality

reduction and capturing latent volatility regimes.

I.3.5 VolatilityVAE_LSTM

```
class VolatilityVAE_LSTM(nn.Module):
    def __init__(self, input_dim, seq_len, latent_dim=10, hidden_dim=64, beta=1.0):
        super().__init__()
        self.input_dim = input_dim
        self.seq_len = seq_len
        self.latent_dim = latent_dim
        self.hidden_dim = hidden_dim
        self.beta = beta

        # Encoder: LSTM -> (mu, logvar)
        self.encoder_lstm = nn.LSTM(input_dim, hidden_dim, batch_first=True)
        self.mu_layer = nn.Linear(hidden_dim, latent_dim)
        self.logvar_layer = nn.Linear(hidden_dim, latent_dim)

        # Decoder: latent z -> initial hidden state     reconstructed sequence
        self.decoder_init = nn.Linear(latent_dim, hidden_dim)
        self.decoder_lstm = nn.LSTM(input_dim, hidden_dim, batch_first=True)
        self.output_layer = nn.Linear(hidden_dim, input_dim)

    def encode(self, x):
        _, (h_n, _) = self.encoder_lstm(x)  # h_n: (1, batch, hidden_dim)
        h_n = h_n.squeeze(0)
        mu = self.mu_layer(h_n)
        logvar = self.logvar_layer(h_n)
        return mu, logvar

    def reparameterize(self, mu, logvar):
        [...]

    def decode(self, z, seq_len=None):
        if seq_len is None:
            seq_len = self.seq_len

        batch_size = z.size(0)
        h_0 = torch.tanh(self.decoder_init(z)).unsqueeze(0)  # (1, batch, hidden_dim)
        c_0 = torch.zeros_like(h_0)

        # Feed zeros as inputs for decoding (teacher forcing can be added later)
        decoder_input = torch.zeros((batch_size, seq_len, self.input_dim), device=z.
            device)
        decoder_output, _ = self.decoder_lstm(decoder_input, (h_0, c_0))
        recon = self.output_layer(decoder_output)
        return recon

    def forward(self, x):
        [...]

    def compute_loss(self, x, recon, mu, logvar):
        [...]
```

The `VolatilityVAE_LSTM` extends the Flat VAE by replacing the fully connected encoder/decoder with LSTM layers, which capture temporal dependencies in volatility sequences. The encoder LSTM compresses the input into a hidden state, used to compute $(\mu, \log \sigma^2)$, while the decoder LSTM reconstructs the sequence step by step from the latent vector. This design makes it more suitable for sequential financial data, where order and memory of past observations matter.

I.4 Train/Test Split for Time Series

Before training our VAE models, we split the dataset into a **training** and a **test** set. Since we are working with time series, we apply a **chronological split** without shuffling: the first 80% of samples are used for training, and the remaining 20% for testing. This setup ensures that the model only learns from past data, respecting the temporal order.

```
from torch.utils.data import TensorDataset, DataLoader

# Convert NumPy arrays to PyTorch tensors
X_tensor = torch.tensor(X_scaled, dtype=torch.float32)
y_tensor = torch.tensor(y_scaled, dtype=torch.float32)

# Define split index
split_ratio = 0.8
split_idx = int(len(X_tensor) * split_ratio)

# Chronological split (no shuffle)
X_train, X_test = X_tensor[:split_idx], X_tensor[split_idx:]
y_train, y_test = y_tensor[:split_idx], y_tensor[split_idx:]

# Wrap into datasets
train_dataset = TensorDataset(X_train, y_train)
test_dataset = TensorDataset(X_test, y_test)

# Dataloaders
batch_size = 64
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

print("Train set shape:", X_train.shape)
print("Test set shape :", X_test.shape)
```

Code Explanation. This block prepares the data for neural network training. First, the scaled input and target arrays are converted into PyTorch tensors. The dataset is then split chronologically into training (first 80%) and test (last 20%) sets, which is crucial for time series tasks where shuffling would break temporal order. These subsets are wrapped into `TensorDataset` objects, which pair each input sequence X_t with its target y_{t+1}. Finally, `DataLoader` structures are created to feed the model in mini-batches of size 64. This batching mechanism is essential in deep learning: it improves computational efficiency on GPUs, stabilizes gradient estimation through stochastic optimization, and ensures that the network learns from manageable chunks of the data while preserving sequence order.

I.5 Training the Model

We now proceed to train the VAE models introduced earlier using the prepared train/test split. The training loop is structured to optimize the total loss, which combines reconstruction accuracy and KL regularization, to monitor convergence over epochs for both train and test sets, and to extract and visualize the learned latent representations z. We use a batch size of 64, the Adam optimizer, a learning rate of 1e−3, and no shuffling to preserve time order.

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt

def train_vae(model, train_loader, test_loader,
              n_epochs=50, lr=1e-3, device="cuda" if torch.cuda.is_available() else "
                  cpu"):
    model.to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    train_losses, val_losses = [], []
    train_recons, val_recons = [], []
    train_kls, val_kls = [], []

    for epoch in range(n_epochs):
        model.train()
```

```
        total_loss, recon_loss, kl_loss = 0, 0, 0

        for x_batch, _ in train_loader:
            x_batch = x_batch.to(device)

            recon, mu, logvar = model(x_batch)
            loss, recon_l, kl_l = model.compute_loss(x_batch, recon, mu, logvar)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            recon_loss += recon_l.item()
            kl_loss += kl_l.item()

        # Average per epoch
        total_loss /= len(train_loader)
        recon_loss /= len(train_loader)
        kl_loss /= len(train_loader)

        train_losses.append(total_loss)
        train_recons.append(recon_loss)
        train_kls.append(kl_loss)

        # Evaluation on test set
        model.eval()
        with torch.no_grad():
            val_loss, val_recon, val_kl = 0, 0, 0
            for x_batch, _ in test_loader:
                x_batch = x_batch.to(device)
                recon, mu, logvar = model(x_batch)
                loss, recon_l, kl_l = model.compute_loss(x_batch, recon, mu, logvar)
                val_loss += loss.item()
                val_recon += recon_l.item()
                val_kl += kl_l.item()

            val_loss /= len(test_loader)
            val_recon /= len(test_loader)
            val_kl /= len(test_loader)

            val_losses.append(val_loss)
            val_recons.append(val_recon)
            val_kls.append(val_kl)

        # Logging
        print(f"Epoch {epoch+1}/{n_epochs} | "
              f"Train Loss: {total_loss:.4f} (Recon: {recon_loss:.4f}, KL: {kl_loss:.4
                  f}) | "
              f"Val Loss: {val_loss:.4f} (Recon: {val_recon:.4f}, KL: {val_kl:.4f})")

    return {
        "train_loss": train_losses,
        "val_loss": val_losses,
        "train_recon": train_recons,
        "val_recon": val_recons,
        "train_kl": train_kls,
        "val_kl": val_kls,
    }
```

I.5.1 Convergence Curves

```
def compare_training_losses(flat_hist, lstm_hist):
    epochs = range(1, len(flat_hist["train_loss"]) + 1)

    plt.figure(figsize=(15, 4))

    # Total Loss
```

```
    plt.subplot(1, 3, 1)
    plt.plot(epochs, flat_hist["val_loss"], label="Flat VAE", color='blue')
    plt.plot(epochs, lstm_hist["val_loss"], label="LSTM VAE", color='purple')
    plt.title("Validation     Total Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()

    # Reconstruction
    plt.subplot(1, 3, 2)
    plt.plot(epochs, flat_hist["val_recon"], label="Flat VAE", color='blue')
    plt.plot(epochs, lstm_hist["val_recon"], label="LSTM VAE", color='purple')
    plt.title("Validation     Reconstruction Loss")
    plt.xlabel("Epoch")
    plt.ylabel("MSE")
    plt.legend()

    # KL Divergence
    plt.subplot(1, 3, 3)
    plt.plot(epochs, flat_hist["val_kl"], label="Flat VAE", color='blue')
    plt.plot(epochs, lstm_hist["val_kl"], label="LSTM VAE", color='purple')
    plt.title("Validation     KL Divergence")
    plt.xlabel("Epoch")
    plt.ylabel("KL")
    plt.legend()

    plt.tight_layout()
    plt.show()
```

I.5.2 First Training Session

We begin by running a first training session using common hyperparameters. Both the FlatVAE and LSTMVAE are trained for 50 epochs, with a latent dimension of 10, a hidden dimension of 64, $\beta = 0.2$, and the Adam optimizer with a learning rate of 1e−3.

```
# Common training params
params = {
    "input_dim": X_scaled.shape[2],
    "seq_len": X_scaled.shape[1],
    "latent_dim": 10,
    "hidden_dim": 64,
    "beta": 0.2,
    "n_epochs": 50,
    "lr": 1e-3,
    "device": "cuda" if torch.cuda.is_available() else "cpu"
}
print(params)

# Train Flat VAE
print("Training VolatilityVAE_Flat...\n")
vae_flat = VolatilityVAE_Flat(
    input_dim=params["input_dim"],
    seq_len=params["seq_len"],
    latent_dim=params["latent_dim"],
    hidden_dim=params["hidden_dim"],
    beta=params["beta"]
)

flat_history = train_vae(
    vae_flat,
    train_loader,
    test_loader,
    n_epochs=params["n_epochs"],
    lr=params["lr"],
    device=params["device"]
)

# Train LSTM VAE
[...]
```

```
compare_training_losses(flat_history, lstm_history)
```

The following figure illustrates the comparison of training curves between the two models.

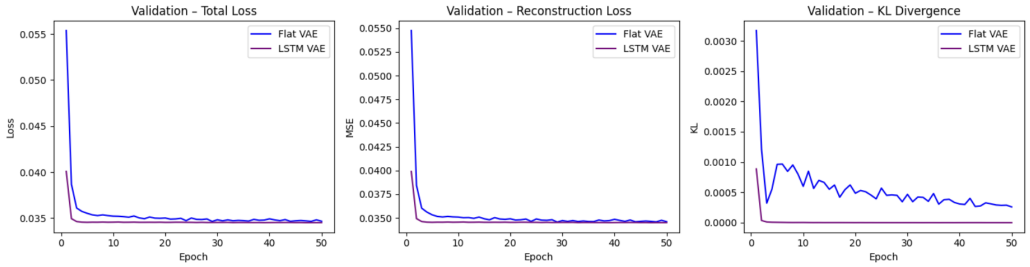


Figure 1: Comparison of convergence between FlatVAE and LSTMVAE on training and validation losses.

I.6 Forecasting Volatility at $T + 1$

Now that our Variational Autoencoders (VAE) — both Flat and LSTM-based — have been trained to extract meaningful latent representations of past market dynamics, we move to the final predictive step of our modeling pipeline: can the latent code z, sampled from past market conditions, help us predict the realized volatility at $T + 1$?

This section focuses on building a regression head that maps the learned latent vector z to the target volatility. The idea is to treat the encoder part of the VAE as a feature extractor, and to add a supervised module trained to minimize the error between predicted and actual volatility.

What we will do:

1. Freeze the trained VAE encoder and extract latent vectors z for all sequences.
2. Build a lightweight regression head $z \to \text{vol}(T + 1)$, trained on the same train/test split.
3. Evaluate the prediction using metrics such as Mean Squared Error (MSE), Mean Absolute Error (MAE), and Pearson correlation (ρ), as well as visual comparison between predicted and realized volatility.
4. Optionally, break down results by market regime (Bull/Bear).

This stage bridges the gap between unsupervised latent learning and supervised forecasting, allowing us to quantify how much predictive signal the VAE has retained from the historical data.

I.6.1 Training $z \to \text{vol}(T + 1)$

We train a simple MLP head on the latent variables extracted from each model (FlatVAE and LSTMVAE).

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.metrics import mean_squared_error, mean_absolute_error
from scipy.stats import pearsonr
import matplotlib.pyplot as plt

# Define a small regression head
class LatentToVol(nn.Module):
    def __init__(self, latent_dim):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(latent_dim, 16),
            nn.ReLU(),
            nn.Linear(16, 1)
```

```
        )

    def forward(self, z):
        return self.model(z)

# Helper function to extract z for a given model and dataset
def extract_latents(vae_model, data_loader, device):
    vae_model.eval()
    zs = []
    ys = []
    with torch.no_grad():
        for x_batch, y_batch in data_loader:
            x_batch = x_batch.to(device)
            mu, _ = vae_model.encode(x_batch)
            zs.append(mu.cpu())
            ys.append(y_batch)
    return torch.cat(zs), torch.cat(ys)

# Training function for z     vol(T+1)
def train_latent_regressor(z, y, latent_dim, n_epochs=100, lr=1e-3):
    model = LatentToVol(latent_dim)
    optimizer = optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.MSELoss()

    for epoch in range(n_epochs):
        model.train()
        optimizer.zero_grad()
        preds = model(z)
        loss = loss_fn(preds, y)
        loss.backward()
        optimizer.step()

        if epoch % 10 == 0:
            print(f"[Epoch {epoch}] Loss: {loss.item():.5f}")

    return model
```

I.6.2 Performance Evaluation

After training, we predict on the test set and calculate several metrics, followed by graphical comparisons.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import pearsonr


def evaluate_latent_regressor(model, z_test, y_test, title="Forecast vs Realized
    Volatility"):
    model.eval()
    with torch.no_grad():
        preds = model(z_test).squeeze().numpy()
        true = y_test.squeeze().numpy()

    # Metrics
    rmse = np.sqrt(np.mean((true - preds) ** 2))
    nrmse = rmse / np.std(true)
    corr, _ = pearsonr(true, preds)

    true_diff = np.diff(true)
    pred_diff = np.diff(preds)
    correct_direction = np.sign(true_diff) == np.sign(pred_diff)
    direction_acc = np.mean(correct_direction)

    print(f" Normalized RMSE: {nrmse:.5f}")
    print(f" Pearson Correlation: {corr:.4f}")
    print(f" Directional Accuracy: {direction_acc*100:.2f}%")

    # Plot 1: Forecast vs Realized
    plt.figure(figsize=(14, 5))
```

```
    plt.plot(true[-300:], label="Realized Vol (T+1)", linewidth=2)
    plt.plot(preds[-300:], label="Predicted Vol (T+1)", linewidth=2, linestyle='--')
    plt.title(title)
    plt.xlabel("Time")
    plt.ylabel("Scaled Volatility")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

    # Plot 2: Volatility variation
    plt.figure(figsize=(14, 4))
    plt.plot(true_diff[-299:], label="   Realized Vol", color="blue")
    plt.plot(pred_diff[-299:], label="   Predicted Vol", color="orange")
    plt.axhline(0, color='gray', linestyle='--')
    plt.title("Volatility Variation Forecast (   t+1)")
    plt.xlabel("Time")
    plt.ylabel("   Scaled Volatility")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```

I.6.3 Evaluation of Models

We test the latent regression performance for both the FlatVAE and the LSTMVAE.

```
# 1. Extract latent vectors
z_flat, y_flat = extract_latents(vae_flat, test_loader, params['device'])
z_lstm, y_lstm = extract_latents(vae_lstm, test_loader, params['device'])

# 2. Train regression heads
reg_flat = train_latent_regressor(z_flat, y_flat, params['latent_dim'])
reg_lstm = train_latent_regressor(z_lstm, y_lstm, params['latent_dim'])

# 3. Evaluate
evaluate_latent_regressor(reg_flat, z_flat, y_flat, title="Flat VAE Forecast")
evaluate_latent_regressor(reg_lstm, z_lstm, y_lstm, title="LSTM VAE Forecast")
```

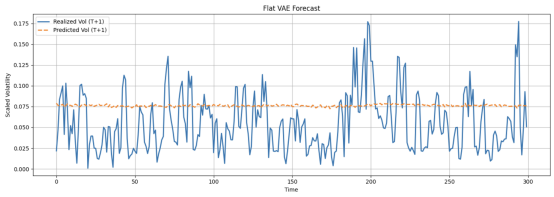


Figure 2: FlatVAE forecast vs realized volatility.

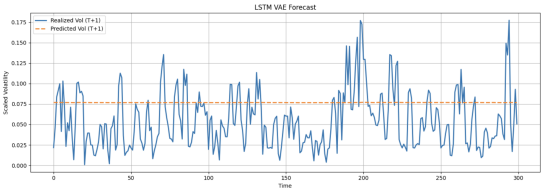


Figure 3: LSTMVAE forecast vs realized volatility.

Limitations and Next Steps. While encouraging, these initial results are still not sufficient for practical deployment. The predictive accuracy remains limited, especially in extreme volatility regimes. This motivates the exploration of more advanced architectures, capable of better capturing the complex temporal dynamics of volatility. In the following sections, we will investigate several extensions of the GRU-VAE framework to improve forecasting power.

II Part II — Volatility Forecasting with VAE-GRU

Having explored the modeling of volatility regimes through unsupervised learning using LSTM-based Variational Autoencoders (VAE), this second part of the project shifts the focus toward **direct forecasting of future volatility** (e.g., at $T+1$ or $T+3$) using a **VAE-GRU architecture**.

Objective

The aim is now to **predict realized volatility at $T+1$ (or a future horizon $T+n$)** directly from past sequences of market features, using a probabilistic model that captures latent structures while leveraging the temporal dynamics of the data.

We explore a more **sequence-to-target** architecture: the encoder ingests past windows (e.g., 20 timesteps), learns a latent representation, and the decoder maps it to a single scalar volatility forecast. This marks a shift away from pure reconstruction toward **probabilistic supervised forecasting**.

Why GRU?

GRUs (Gated Recurrent Units) offer a simpler and often more stable alternative to LSTMs for sequence encoding. They tend to:

- Require fewer parameters and training time,
- Be less prone to overfitting on small sequences,
- Capture medium-term dependencies effectively (useful for volatility windows like 3-day or 5-day rolling standard deviation).

This makes GRUs attractive when the goal is to compress recent market movements into a compact latent variable for forecasting.

Theoretical Framework

A GRU processes sequential data with two gates (update z_t and reset r_t). Given input x_t and previous hidden state h_{t-1}:

$$
\begin{aligned}
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z), \quad \sigma(x) = \frac{1}{1+e^{-x}} && \text{(update gate)} \\
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) && \text{(reset gate)} \\
\tilde{h}_t &= \tanh\big(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h\big), \quad \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} && \text{(candidate state)} \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t && \text{(new hidden state)}
\end{aligned}
$$

The hidden state h_t summarizes all past information $\{x_{t-N+1}, \ldots, x_t\}$ into a compact vector representation.

We adopt a **VAE formulation for time series**, where the **encoder** is a GRU and the **decoder** is a feedforward regressor. The encoder compresses historical sequences into a latent variable z_t, and this latent code is then used to predict the next-step realized volatility.

Key Formulations:

$$
\begin{aligned}
h_t &= \text{GRU}(x_{t-N+1}, \ldots, x_t) \\
\mu_z, \log \sigma_z &= f_{\text{enc}}(h_t) \\
z_t &= \mu_z + \sigma_z \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \\
\hat{y}_{t+1} &= f_{\text{dec}}(z_t)
\end{aligned}
$$

Here: - h_t is the hidden state encoding the input window up to time t, - $\mu_z, \log \sigma_z$ are the parameters of the latent Gaussian distribution produced by the encoder, - z_t is the sampled latent representation, - $\hat{y}_{t+1}$ denotes the **predicted realized volatility at $t+1$**.

Loss Function.

$$\mathcal{L} = E_{q(z|x)}\left[(y_{t+1} - \hat{y}_{t+1})^2\right] + \beta \cdot D_{KL}\big(q(z|x) \,\|\, p(z)\big)$$

The first term enforces accurate prediction of future volatility, while the second term (the KL divergence) regularizes the latent distribution so that it remains close to the prior $p(z) = \mathcal{N}(0, I)$, ensuring stability and generalization.

Dataset Construction

We redefine the input/output pairs as follows:

- **Input sequences:** past windows of size N (e.g., 20 timesteps) containing returns, VIX, rolling volatility, etc.
- **Target:** realized volatility at $T + 1$, depending on the experiment.

Data is scaled and batched identically to Part I for consistency.

Comparison with Part I:

Part I (Reconstruction)	**Part II (Forecasting)**
Decoder reconstructs X_t	Decoder predicts y_{t+H}
Loss: reconstruction + KL	Loss: forecast error + KL
Output: full sequence	Output: scalar volatility forecast

II.1 GRU-based VAE Model

Architecture: GRU Encoder + Regressor Decoder. We now design a **Variational Autoencoder (VAE)** where the **encoder** is a GRU network that learns to **summarize a sequence of market features** into a latent distribution $(\mu, \log \sigma^2)$. The **latent space** captures uncertainty and market regimes, and the **decoder** is a feed-forward regressor that maps latent vectors $z \sim \mathcal{N}(\mu, \sigma^2)$ to **future volatility estimates**.

Architecture Overview.

1. **Encoder (GRU):**
$$x_{1:T} \longrightarrow \text{GRU} \longrightarrow h_T \longrightarrow (\mu, \log \sigma^2)$$
2. **Latent Sampling:**
$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$
3. **Decoder (Regressor):**
$$z \longrightarrow \text{MLP} \longrightarrow \hat{y}_{t+H}$$

   Here, the **MLP (Multi-Layer Perceptron)** is a sequence of fully connected layers. A single perceptron layer applies an affine transformation followed by a non-linearity:

   $$h_1 = \sigma\big(W_1 z + b_1\big),$$

   where $W_1 \in R^{m \times k}$ and $b_1 \in R^m$ are trainable parameters, and $\sigma(\cdot)$ is a non-linear activation (e.g., ReLU or tanh).

   Stacking several such layers yields the MLP:

   $$h_\ell = \sigma\big(W_\ell h_{\ell-1} + b_\ell\big), \quad \ell = 2, \ldots, L,$$

   and the final prediction is obtained by a last linear projection:

   $$\hat{y}_{t+H} = W_{L+1} h_L + b_{L+1}.$$

Loss Function. The total loss is a combination of **reconstruction loss** (MSE between predicted and true volatility) and **KL divergence** between the latent distribution and the standard Gaussian prior:

$$\mathcal{L}_{\text{VAE}} = \text{MSE}(\hat{y}, y) + \beta \cdot D_{KL}[q(z|x) \parallel \mathcal{N}(0, I)] \,.$$

II.2 GRU-based VAE Implementation

PyTorch Implementation. The following code defines the `VAE_GRU_Volatility` class, which combines a GRU encoder with a regressor decoder. The encoder compresses past market features into a latent distribution $(\mu, \log \sigma^2)$, the reparameterization trick samples latent vectors, and the decoder predicts realized volatility at horizon $T + H$.

Listing 1: GRU-based VAE model for volatility forecasting

```
class VAE_GRU_Volatility(nn.Module):
    def __init__(self, input_dim, seq_len, hidden_dim, latent_dim, beta=0.2):
        super().__init__()
        self.input_dim = input_dim
        self.seq_len = seq_len
        self.hidden_dim = hidden_dim
        self.latent_dim = latent_dim
        self.beta = beta

        # Encoder
        self.encoder_rnn = nn.GRU(input_dim, hidden_dim, batch_first=True)
        self.mu_layer = nn.Linear(hidden_dim, latent_dim)
        self.logvar_layer = nn.Linear(hidden_dim, latent_dim)

        # Decoder (regressor head)
        self.decoder_mlp = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1)
        )

    def encode(self, x):
        _, h = self.encoder_rnn(x)          # h: [1, batch, hidden_dim]
        h = h.squeeze(0)                    # [batch, hidden_dim]
        mu = self.mu_layer(h)               # [batch, latent_dim]
        logvar = self.logvar_layer(h)       # [batch, latent_dim]
        return mu, logvar

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        return self.decoder_mlp(z)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        y_hat = self.decode(z)
        return y_hat, mu, logvar

    def loss_function(self, y_hat, y_true, mu, logvar):
        # Reconstruction loss (MSE)
        recon_loss = nn.functional.mse_loss(y_hat, y_true, reduction='mean')

        # KL divergence
        kl_div = -0.5 * torch.mean(1 + logvar - mu.pow(2) - logvar.exp())

        # Total VAE loss
        return recon_loss + self.beta * kl_div, recon_loss, kl_div
```

Code Explanation. This class implements a GRU-based Variational Autoencoder (VAE) specifically designed for volatility forecasting. The encoder is a GRU that summarizes the input time series

into a hidden state, which is then mapped to the mean μ and log-variance $\log \sigma^2$ of the latent distribution. The *reparameterization trick* samples latent vectors z in a differentiable way. The decoder is a simple MLP that transforms z into a forecast $\hat{y}$ of future volatility. The loss function combines a reconstruction error (MSE between $\hat{y}$ and the true volatility) with a KL divergence term that regularizes the latent distribution toward a standard Gaussian prior. This design allows the model both to capture temporal dependencies (via GRU) and to learn a structured latent representation that balances accuracy and generative regularization.

$$x_{1:T} \xrightarrow{\text{GRU}} h_T \xrightarrow{\mu, \log \sigma^2} z = \mu + \sigma \cdot \epsilon \xrightarrow{\text{MLP}} \hat{y}_{t+1}$$

II.3 Training Setup

We now train the GRU-based VAE model to **forecast volatility at $T+1$ (or $T+3$)** directly from historical market behavior.

Training Details.

- **Input:** Sequences of shape (batch_size, seq_len, input_dim)
- **Target:** Scaled realized volatility at $T+H$
- **Loss:** MSE (forecast accuracy) + KL divergence with $\mathcal{N}(0, I)$
- **Optimizer:** Adam
- **Device:** CPU or GPU (automatically selected)
- **Metrics:** Total VAE loss, KL divergence, MSE

Listing 2: Training function for GRU-VAE

```
def train_gru_vae(model, train_loader, test_loader, n_epochs=50, lr=1e-3, device="cpu"
    ):
    model.to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    train_losses, test_losses = [], []

    for epoch in range(n_epochs):
        model.train()
        train_epoch_loss = 0.0

        for X_batch, y_batch in train_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            optimizer.zero_grad()
            y_hat, mu, logvar = model(X_batch)
            loss, recon, kl = model.loss_function(y_hat, y_batch, mu, logvar)
            loss.backward()
            optimizer.step()

            train_epoch_loss += loss.item()

        train_epoch_loss /= len(train_loader)
        train_losses.append(train_epoch_loss)

        # Validation
        model.eval()
        val_loss = 0.0
        with torch.no_grad():
            for X_val, y_val in test_loader:
                X_val = X_val.to(device)
                y_val = y_val.to(device)
                y_hat, mu, logvar = model(X_val)
                loss, _, _ = model.loss_function(y_hat, y_val, mu, logvar)
```

```
                val_loss += loss.item()

        val_loss /= len(test_loader)
        test_losses.append(val_loss)

        print(f"Epoch {epoch+1}/{n_epochs} | Train Loss: {train_epoch_loss:.4f} | Val
            Loss: {val_loss:.4f}")

    return {"train": train_losses, "val": test_losses}
```

II.4 Evaluation

Forecasting Performance. Now that our GRU-based VAE is trained, we evaluate its **forecasting ability** on unseen test data. We focus on three key metrics:

- **Normalized RMSE (NRMSE):**

$$\text{NRMSE} = \frac{\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}}{\text{std}(y)}$$

- **Pearson Correlation:**

$$\rho(y, \hat{y}) = \frac{\text{Cov}(y, \hat{y})}{\sigma_y \, \sigma_{\hat{y}}}, \quad \text{with } \text{Cov}(y, \hat{y}) = \frac{1}{n-1}\sum_{i=1}^{n}(y_i - \bar{y})\big(\hat{y}_i - \bar{\hat{y}}\big)$$

- **Directional Accuracy:**

$$\text{Accuracy} = \frac{1}{n-1}\sum_{i=2}^{n} 1_{\text{sign}(y_i - y_{i-1}) = \text{sign}(\hat{y}_i - \hat{y}_{i-1})}$$

Implementation. Listing 3: Evaluation of GRU-VAE forecasting performance

```
from scipy.stats import pearsonr

def evaluate_forecast(model, test_loader, device="cpu", title="GRU-VAE Volatility
    Forecast"):
    model.eval()
    preds, true = [], []

    with torch.no_grad():
        for X_batch, y_batch in test_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            y_hat, _, _ = model(X_batch)
            preds.append(y_hat.cpu().numpy())
            true.append(y_batch.cpu().numpy())

    preds = np.concatenate(preds).squeeze()
    true = np.concatenate(true).squeeze()

    # Metrics
    rmse = np.sqrt(np.mean((true - preds)**2))
    nrmse = rmse / np.std(true)
    corr, _ = pearsonr(true, preds)

    true_diff = np.diff(true)
    pred_diff = np.diff(preds)
    direction_acc = np.mean(np.sign(true_diff) == np.sign(pred_diff))

    print(f"Normalized RMSE:       {nrmse:.5f}")
    print(f" Pearson Correlation:  {corr:.4f}")
    print(f" Directional Accuracy: {direction_acc*100:.2f}%")

    # Plot 1
    [Reuse previous evaluation blocks]
```

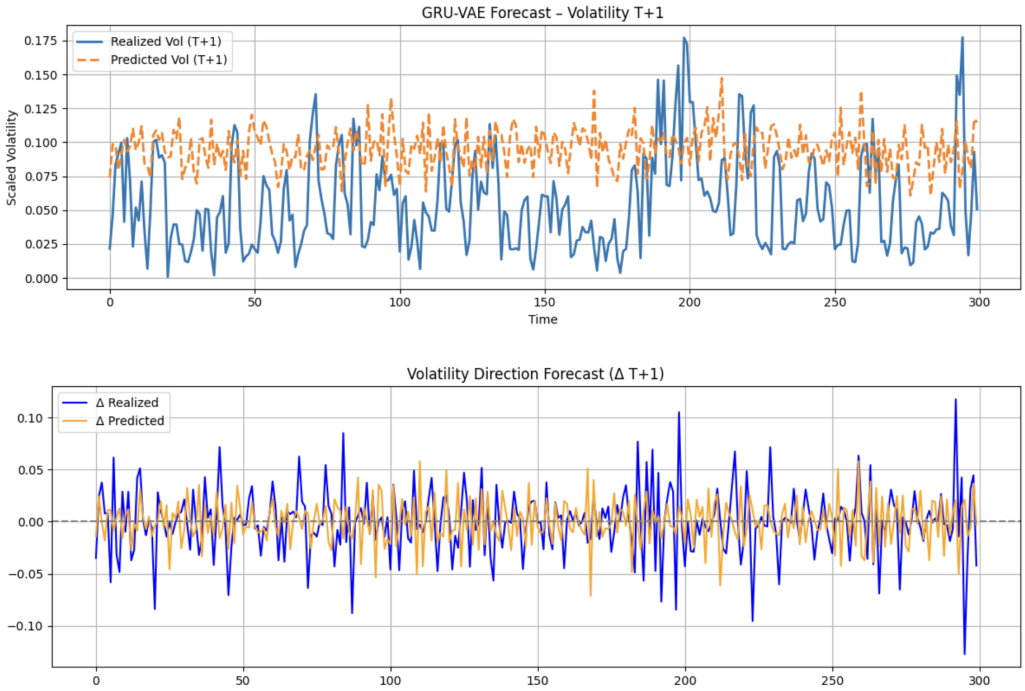


Figure 4: Comparison of Volatility forecast at T+1 and reality and comparison within true and predicted diractions of Volatility.

Commentary. The results obtained so far show that we have made significant progress: the GRU-based VAE is finally able to generate a genuine predictive signal, rather than overfitting to a simple moving average as observed with the LSTM baseline. The model effectively captures temporal dependencies in volatility and avoids trivial reconstructions. It is important to recall that the two main points of model efficiency are the **detection of volatility peaks** and the **amplitude of these peaks** in order to match real market behavior. The next step will be to explore techniques to **denoise the signal**, in order to enhance both the robustness and the interpretability of forecasts.

II.5 Direction-Aware GRU-VAE: Custom Loss Design

Motivation. In volatility forecasting—especially for financial risk metrics such as realized volatility at $T+1$—it is often more valuable to predict the **direction of volatility changes** than to estimate the exact value. Standard losses such as MSE tend to penalize magnitude errors heavily, even if the direction of volatility movement is correct. This misalignment with practical needs in **risk management, volatility targeting, and hedging** motivates the design of a custom, direction-aware loss.

We therefore introduce a **hybrid loss** in our GRU-based VAE that:

- Maintains continuous prediction of volatility,
- Adds directional awareness to the objective,
- Balances precision and interpretability.

Architecture Recap. We build a standard VAE with:

- A GRU encoder that embeds past sequences into a latent space,
- Latent sampling with μ and σ^2,
- An MLP decoder producing the forecasted (scaled) volatility at $T+H$,
- An auxiliary classification head to predict the next **directional movement** of volatility.

Custom Loss Function. Our loss combines three components:

1. **Reconstruction Loss (MSE):**

$$\mathcal{L}_{\text{recon}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

2. **KL Divergence:**

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2}\sum_{j=1}^{d}\left(1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2\right)$$

   where μ_j, σ_j^2 are the mean and variance of the log-latent representation.

3. **Directional Classification Loss (BCE):**

$$l_i = \begin{cases} 1 & \text{if } y_i > y_{i-1} \quad \text{(volatility up)} \\ 0 & \text{otherwise} \quad \text{(down or flat)} \end{cases}$$

$$\mathcal{L}_{\text{dir}} = -\frac{1}{N}\sum_{i=1}^{N}\left[l_i \log(p_i) + (1 - l_i)\log(1 - p_i)\right]$$

   where $p_i = \sigma(\text{logits}_i)$ is the predicted probability.

Final Loss. The total training loss is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \beta \cdot \mathcal{L}_{\text{KL}} + \gamma \cdot \mathcal{L}_{\text{dir}}$$

where β controls KL scaling and γ controls the importance of directional classification.

Listing 4: Direction-Aware GRU-VAE with custom hybrid loss

Implementation.

```
class VAE_GRU_LogNormal(nn.Module):
    def __init__(self, input_dim, seq_len, hidden_dim, latent_dim, beta=0.2, gamma
        =0.5):
        super().__init__()
        self.encoder_rnn = nn.GRU(input_dim, hidden_dim, batch_first=True)
        self.mu_layer = nn.Linear(hidden_dim, latent_dim)
        self.logvar_layer = nn.Linear(hidden_dim, latent_dim)
        self.decoder_mlp = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1)
        )
        self.beta = beta
        self.gamma = gamma

    def encode(self, x):
        _, h = self.encoder_rnn(x)
        h = h.squeeze(0)
        mu = self.mu_layer(h)
        logvar = self.logvar_layer(h)
        return mu, logvar

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        log_z = mu + eps * std
        return torch.exp(log_z)  # LogNormal sample

    def decode(self, z):
        return self.decoder_mlp(z)

    def forward(self, x):
```

```
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        y_hat = self.decode(z)
        return y_hat, mu, logvar

    def loss_function(self, y_hat, y_true, mu, logvar):
        recon_loss = F.mse_loss(y_hat, y_true, reduction='mean')
        kl = 0.5 * torch.sum(torch.exp(logvar) + mu**2 - 1 - logvar) / mu.size(0)
        with torch.no_grad():
            y_diff = torch.diff(y_true, dim=0)
            y_label = (y_diff > 0).float()
        pred_diff = torch.diff(y_hat, dim=0)
        pred_probs = torch.sigmoid(pred_diff)
        class_loss = F.binary_cross_entropy(pred_probs, y_label)
        total_loss = recon_loss + self.beta * kl + self.gamma * class_loss
        return total_loss, recon_loss, kl, class_loss
```

Results. The inclusion of a direction-aware classification signal allows the model to:

- Retain high-quality volatility magnitude predictions,
- Improve **directional accuracy**, crucial for trading and hedging,
- Deliver more robust performance even under noisy, scaled features.

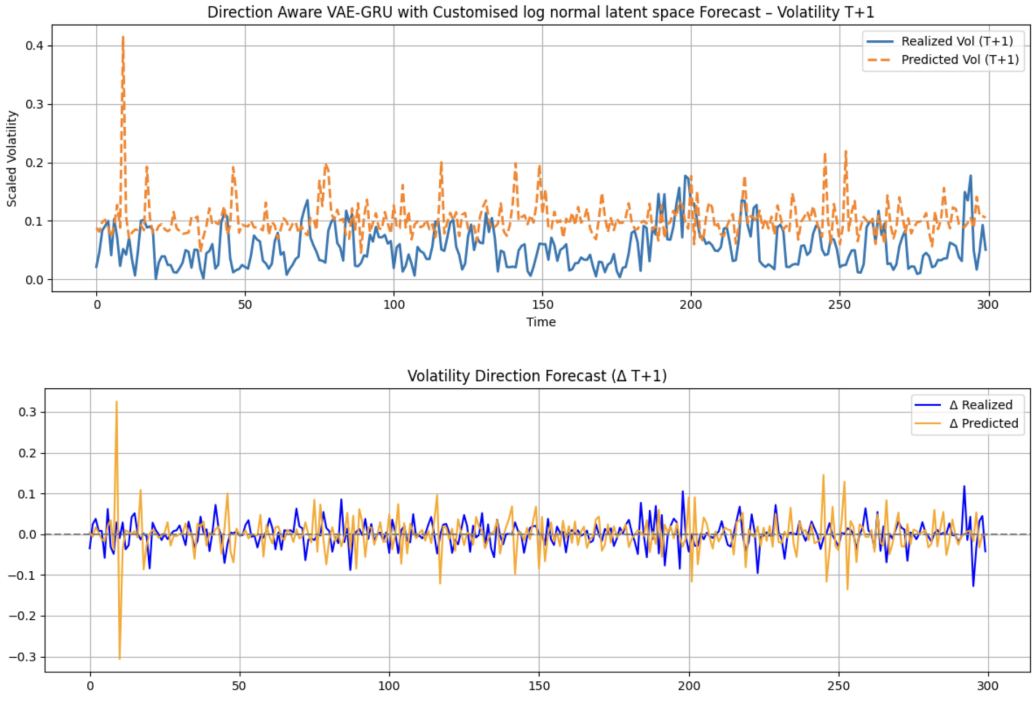


Figure 5: Comparison of Volatility forecast at T+1 and reality and comparison within true and predicted diractions of Volatility.

Commentary 1. The integration of a log-normal latent space combined with a cross-entropy directional term clearly improves the detection of volatility spikes. However, the forecasts exhibit a systematic shift, with predicted amplitudes often exceeding the true magnitude of realized volatility. This suggests that while peak timing is better captured, the calibration of amplitude remains problematic.

II.6 Fourier Loss Integration in VAE-GRU

Motivation & Transition. To better model the dynamic and irregular nature of volatility—often characterized by sudden spikes and regime shifts—we extend the sequential VAE-GRU architecture by incorporating a **Fourier-based spectral loss**. Rather than modifying the model structure, we enrich the **loss function** to penalize discrepancies not only in the time domain (via MSE), but also in the **frequency domain**, encouraging the model to retain the spectral identity of the true signal.

Combined Loss Function: Time + Frequency. The total loss integrates three terms:

$$\mathcal{L}_{\text{total}} = \underbrace{\text{MSE}(y, \hat{y})}_{\text{Reconstruction}} + \beta \cdot \underbrace{D_{\text{KL}}[q(z|x) \parallel \mathcal{N}(0, I)]}_{\text{Latent regularization}} + \gamma \cdot \underbrace{\text{MSE}(|\mathcal{F}(y)|, |\mathcal{F}(\hat{y})|)}_{\text{Spectral alignment}}$$

where:

- $\mathcal{F}(y)$ is the discrete Fourier transform (DFT) of the target sequence,
- $|\mathcal{F}(y)|$ is the amplitude spectrum,
- γ controls the importance of frequency alignment.

This **Fourier Loss** improves the model's ability to capture **short-term bursts and regime changes** by aligning temporal and spectral profiles.

Fourier Loss Intuition. The Fourier Loss forces the model to reproduce the dominant frequencies of the target time series.

The DFT (Discrete Fourier Transform) is defined as:

$$\mathcal{F}(y)_k = \sum_{t=0}^{T-1} y_t \cdot e^{-2i\pi kt/T}, \quad k = 0, \ldots, T-1$$

where:

- i denotes the imaginary unit, defined by $i^2 = -1$,
- the exponential $e^{-2i\pi kt/T}$ encodes sinusoidal oscillations in both real and imaginary parts, corresponding to frequency k/T.

The amplitude spectrum is obtained from the complex Fourier coefficients:

$$|\mathcal{F}(y)_k| = \sqrt{\Re^2\big(\mathcal{F}(y)_k\big) + \Im^2\big(\mathcal{F}(y)_k\big)},$$

where:

- $\Re(\mathcal{F}(y)_k)$ denotes the real part of the coefficient (cosine component) [3],
- $\Im(\mathcal{F}(y)_k)$ denotes the imaginary part (sine component) [3].

Finally, the Fourier loss compares the amplitude spectra of the true and predicted signals:

$$\mathcal{L}_{\text{Fourier}} = \text{MSE}(|\mathcal{F}(y)|, |\mathcal{F}(\hat{y})|).$$

Relevance in Volatility Modeling. Volatility shocks, often triggered by exogenous events, appear as **high-frequency components** in the spectrum. A simple temporal MSE is blind to such dynamics. By introducing a frequency penalty, Fourier Loss enables:

- Better detection of transient regimes,
- Improved tracking of volatility bursts,
- Reduced tendency to produce smoothed forecasts.

Implementation.

Listing 5: GRU-VAE with Fourier Loss integration

```
class VAE_GRU_Fourier(nn.Module):
    def __init__(self, input_dim, seq_len, hidden_dim, latent_dim, beta=0.2, gamma
        =0.5):
        super().__init__()
        self.encoder_rnn = nn.GRU(input_dim, hidden_dim, batch_first=True)
        self.mu_layer = nn.Linear(hidden_dim, latent_dim)
        self.logvar_layer = nn.Linear(hidden_dim, latent_dim)
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1)
        )
        self.beta = beta
        self.gamma = gamma

    def encode(self, x):
        _, h = self.encoder_rnn(x)
        h = h.squeeze(0)
        mu = self.mu_layer(h)
        logvar = self.logvar_layer(h)
        return mu, logvar

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        return self.decoder(z)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        y_hat = self.decode(z)
        return y_hat, mu, logvar

    def loss_function(self, y_hat, y_true, mu, logvar):
        # 1. Time-domain reconstruction
        mse_loss = F.mse_loss(y_hat, y_true)

        # 2. Latent regularization
        kl_div = -0.5 * torch.mean(1 + logvar - mu.pow(2) - logvar.exp())

        # 3. Frequency-domain penalty
        fft_true = torch.fft.fft(y_true.squeeze(), dim=0)
        fft_pred = torch.fft.fft(y_hat.squeeze(), dim=0)
        amp_true, amp_pred = torch.abs(fft_true), torch.abs(fft_pred)
        fourier_loss = F.mse_loss(amp_pred, amp_true)

        # Total loss
        total_loss = mse_loss + self.beta * kl_div + self.gamma * fourier_loss
        return total_loss, mse_loss, kl_div, fourier_loss
```

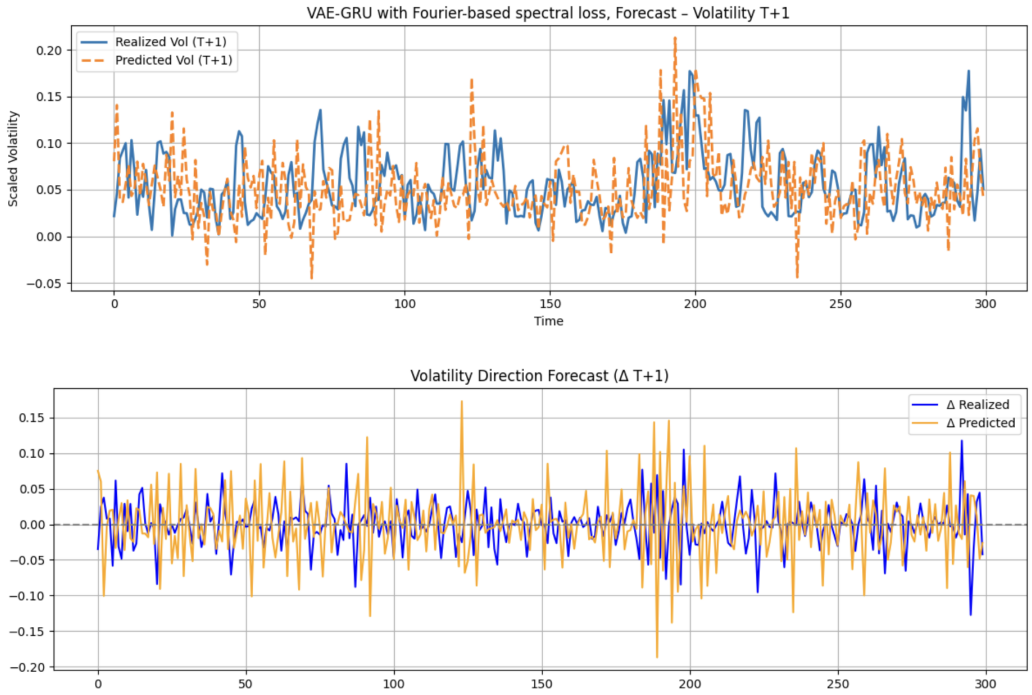


Figure 6: Comparison of Volatility forecast at T+1 and directions vs reality.

Results. The integration of Fourier Loss improves the model's sensitivity to volatility bursts and short-term shocks, aligning forecasts with both temporal and spectral dynamics. The model successfully mitigates the systematic shift observed previously. The overall amplitude alignment with realized volatility is significantly improved. Nonetheless, this comes at the cost of less controlled spike detection, where volatility bursts are sometimes exaggerated or misrepresented. This trade-off highlights the challenge of balancing spectral fidelity with stable predictive accuracy.

III Part III — Transformer + MLP for Volatility Forecasting (T+1)

After experimenting with recurrent architectures (GRU, VAE-GRU) and probabilistic modeling (KL-divergence, log-normal latent spaces), we now explore a **pure attention-based architecture** to predict T+1 volatility.

Motivation

Volatility is inherently noisy and exhibits both short-term shocks and long-range dependencies. Recurrent models often struggle to capture distant dependencies or quickly adapt to regime changes.

Transformers, by contrast:

- leverage **self-attention** to focus on the most relevant parts of the sequence,
- are **parallelizable** and scale efficiently with larger datasets,
- do not rely on strict recurrence, making them well suited for irregular or high-frequency signals.

We therefore combine a **Transformer Encoder** with a lightweight **MLP head** to output the predicted volatility for the next timestep.

—

III.1 Model Architecture: Transformer for Volatility Forecasting

Let the input be a batch of sequences $\mathbf{X} \in R^{B\times T\times d}$, where:

- B: batch size,
- T: sequence length (lookback window),
- d: number of input features.

Step 1: Linear Projection. The raw features are first projected into a model dimension d_{model}:

$$\mathbf{H}_0 = \mathbf{X}\cdot\mathbf{W}_{\text{proj}} + \mathbf{b}_{\text{proj}} \quad \in R^{B\times T\times d_{\text{model}}}.$$

Step 2: Positional Encoding. To preserve temporal order, we add positional encodings:

$$\tilde{\mathbf{H}}_0 = \mathbf{H}_0 + \mathbf{P}.$$

Two versions are possible:

- **Learned Positional Encoding**: $\mathbf{P}$ is trained jointly with the model.
- **Sinusoidal Positional Encoding** (Vaswani et al., 2017) ([5]):

$$\mathbf{P}_{t,2i} = \sin\left(\frac{t}{10000^{2i/d_{\text{model}}}}\right), \quad \mathbf{P}_{t,2i+1} = \cos\left(\frac{t}{10000^{2i/d_{\text{model}}}}\right).$$

Step 3: Transformer Encoder Layer. Each encoder layer applies **multi-head self-attention** followed by a feed-forward network (FFN).

Self-Attention:

$$\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) \;=\; \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}.$$

In this formulation, $\mathbf{Q}$ (queries), $\mathbf{K}$ (keys), and $\mathbf{V}$ (values) are obtained from the same input sequence $X \in R^{T \times d_{\text{model}}}$ through linear projections:

$$\mathbf{Q} = XW^Q, \quad \mathbf{K} = XW^K, \quad \mathbf{V} = XW^V,$$

with $W^Q, W^K, W^V \in R^{d_{\text{model}} \times d_k}$ being learnable parameter matrices. Each row of $\mathbf{Q}$ represents a query vector that encodes the question: *"which other positions should I attend to?"*, while each row of $\mathbf{K}$ is a key vector that defines how relevant each position is. The rows of $\mathbf{V}$ are value vectors, i.e., the actual information to be aggregated.
The product $\mathbf{QK}^\top \in R^{T \times T}$ computes pairwise similarity scores between all positions, which are scaled by $\sqrt{d_k}$ to stabilize gradients. Applying the softmax transforms these scores into attention weights (so that each row sums to one). Finally, multiplication by $\mathbf{V}$ produces a weighted combination of values, meaning that each position in the sequence becomes a contextualized representation that integrates information from the entire sequence.

Multi-Head Attention:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) \cdot W^O.$$

Feed-Forward Network:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2.$$

Each block includes residual connections and layer normalization.

Step 4: Temporal Pooling. After L stacked encoder layers, we obtain contextualized embeddings $\mathbf{H}_L \in R^{B \times T \times d_{\text{model}}}$. We aggregate across time:

$$\bar{\mathbf{h}} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{H}_L[:, t, :].$$

Step 5: Prediction Head (MLP). Finally, volatility at $T+1$ is predicted via:

$$\hat{y} = \text{MLP}(\bar{\mathbf{h}}) = W_2 \cdot \text{ReLU}(W_1 \cdot \bar{\mathbf{h}} + b_1) + b_2.$$

III.2 Transformer + MLP Implementation with Amplified Loss

We implement the proposed Transformer-based volatility forecaster, where a sequence of market features is processed through self-attention layers and pooled into a compact representation, which is then mapped to a one-step volatility forecast via an MLP regression head. To improve peak sensitivity, we introduce an **Amplified MSE Loss**, which increases the penalty around volatility spikes.

```
class VolatilityTransformer(nn.Module):
    def __init__(self, input_dim, seq_len, d_model=64, nhead=4, num_layers=2,
        mlp_hidden=64):
        super().__init__()
        self.seq_len = seq_len
        self.input_dim = input_dim

        # Input projection
        self.input_proj = nn.Linear(input_dim, d_model)

        # Learnable positional encoding
        self.positional_encoding = nn.Parameter(torch.randn(seq_len, d_model))

        # Transformer Encoder
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
```

```
            batch_first=True
        )
        self.transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=
            num_layers)

        # MLP regression head
        self.mlp_head = nn.Sequential(
            nn.Linear(d_model, mlp_hidden),
            nn.ReLU(),
            nn.Linear(mlp_hidden, 1)
        )

    def forward(self, x):
        x = self.input_proj(x)
        x = x + self.positional_encoding.unsqueeze(0)
        x = self.transformer_encoder(x)
        x_pooled = x.mean(dim=1)  # temporal pooling
        return self.mlp_head(x_pooled)


class AmplifiedMSELoss(nn.Module):
    def __init__(self, alpha=2.0):
        super().__init__()
        self.alpha = alpha

    def forward(self, y_pred, y_true):
        weights = 1 + self.alpha * (y_true - y_true.mean()).abs()
        return ((y_pred - y_true) ** 2 * weights).mean()
```

III.3 Training and Evaluation

We train the Transformer with Adam optimizer and evaluate forecasting quality with normalized RMSE, correlation, and directional accuracy.

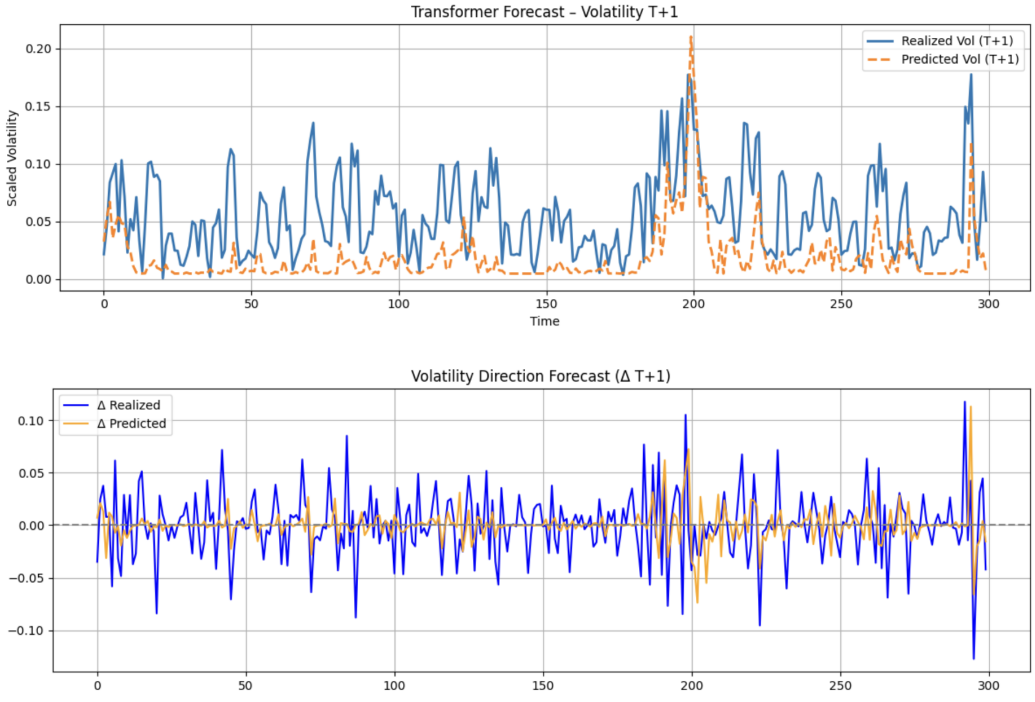


Figure 7: Comparison of Volatility forecast at T+1 and directions vs reality.

Commentary : Baseline Transformer. The baseline Transformer provides an excellent compromise between peak detection and amplitude calibration. However, this increased neural network complexity introduces a new efficiency concern: the current training time reaches nearly 45 minutes for one month of forecasts, raising questions about computational scalability in real-world applications.

III.4 DAVE Transformer Implementation

We extend the baseline Transformer-MLP by introducing a **combined loss** that simultaneously (i) amplifies errors around volatility spikes in the time domain and (ii) aligns the **amplitude spectrum** of predictions and targets in the frequency domain (Fourier).

```
class CombinedLoss(nn.Module):
    def __init__(self, alpha=2.0, gamma=0.5):
        super().__init__()
        self.alpha = alpha  # peak emphasis
        self.gamma = gamma  # Fourier weight

    def forward(self, y_pred, y_true):
        # Time-domain: amplified MSE around peaks
        weights = 1 + self.alpha * (y_true - y_true.mean()).abs()
        mse_weighted = ((y_pred - y_true) ** 2 * weights).mean()

        # Frequency-domain: amplitude-spectrum alignment
        fft_true = torch.fft.fft(y_true.squeeze(), dim=0)
        fft_pred = torch.fft.fft(y_pred.squeeze(), dim=0)
        amp_true = torch.abs(fft_true)
        amp_pred = torch.abs(fft_pred)
        fourier = nn.functional.mse_loss(amp_pred, amp_true)

        return mse_weighted + self.gamma * fourier


def train_transformer_model_v2(model, train_loader, test_loader,
                               n_epochs=50, lr=1e-3, alpha=2.0, gamma=1.0, device="cpu
                                   "):
    model = model.to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    criterion = CombinedLoss(alpha=alpha, gamma=gamma)

    train_losses, test_losses = [], []

    for epoch in range(n_epochs):
        model.train()
        running_loss = 0.0

        for X_batch, y_batch in train_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            optimizer.zero_grad()
            y_pred = model(X_batch)
            loss = criterion(y_pred, y_batch)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()

        train_loss = running_loss / len(train_loader)
        train_losses.append(train_loss)

        # Validation
        model.eval()
        val_loss = 0.0
        with torch.no_grad():
            for X_val, y_val in test_loader:
                X_val = X_val.to(device)
                y_val = y_val.to(device)
                y_pred = model(X_val)
                loss = criterion(y_pred, y_val)
                val_loss += loss.item()
```

```
        test_loss = val_loss / len(test_loader)
        test_losses.append(test_loss)

        print(f"[{epoch+1}/{n_epochs}] Train Loss: {train_loss:.6f} | Test Loss: {
            test_loss:.6f}")

    return {"train_loss": train_losses, "test_loss": test_losses}
```

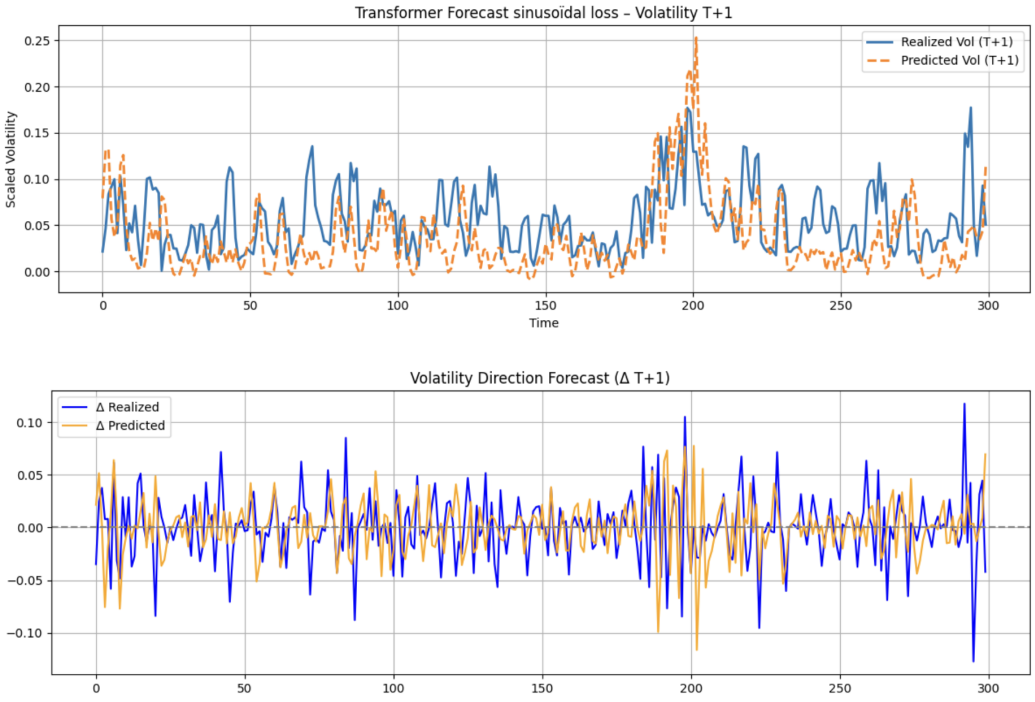


Figure 8: Comparison of Volatility forecast at T+1 and directions vs reality.

Commentary : Fourier-Featured Loss. The Fourier-featured loss encourages forecasts to respect both time-domain peak behavior and frequency content. In practice, this improves overall amplitude calibration while retaining sensitivity to transient regimes; however, careful tuning of (α, γ) is required to avoid over-emphasis on high-frequency components that can destabilize peak detection timing.

III.5 SPECS-Transformer (Sinusoidal Positional Encoding with signal-Spectrum dynamics)

We replace the learnable positional encoding with a **sinusoidal positional encoding** to inject explicit temporal structure, following the classical formulation from attention-based models. This experiment evaluates whether fixed, frequency-based position signals improve the stability of forecasts and the robustness of peak timing.

```
class SinusoidalPositionalEncoding(nn.Module):
    def __init__(self, seq_len, d_model):
        super().__init__()

        position = torch.arange(seq_len).unsqueeze(1)  # [seq_len, 1]
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) /
            d_model))  # [d_model//2]

        pe = torch.zeros(seq_len, d_model)
        pe[:, 0::2] = torch.sin(position * div_term)  # even dims
        pe[:, 1::2] = torch.cos(position * div_term)  # odd dims
```

```
        self.register_buffer("pe", pe)  # non-trainable

    def forward(self, x):
        return x + self.pe.unsqueeze(0)  # [1, seq_len, d_model]


class VolatilityTransformer_sin(nn.Module):
    def __init__(self, input_dim, seq_len, d_model=64, nhead=4, num_layers=2,
        mlp_hidden=64):
        super().__init__()
        self.seq_len = seq_len
        self.input_dim = input_dim

        # Project input features to model dimension
        self.input_proj = nn.Linear(input_dim, d_model)

        # Sinusoidal positional encoding
        self.positional_encoding = SinusoidalPositionalEncoding(seq_len, d_model)

        # Transformer Encoder
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            batch_first=True
        )
        self.transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=
            num_layers)

        # MLP regression head
        self.mlp_head = nn.Sequential(
            nn.Linear(d_model, mlp_hidden),
            nn.ReLU(),
            nn.Linear(mlp_hidden, 1)
        )

    def forward(self, x):
        # x : [batch_size, seq_len, input_dim]
        batch_size, seq_len, input_dim = x.shape
        assert seq_len == self.seq_len, f"Expected seq_len={self.seq_len}, got {
            seq_len}"

        x = self.input_proj(x)  # [batch_size, seq_len, d_model]
        x = x + self.positional_encoding(x)  # add positional encoding
        x = self.transformer_encoder(x)  # [batch_size, seq_len, d_model]
        x_pooled = x.mean(dim=1)  # global average pooling
        return self.mlp_head(x_pooled)  # [batch_size, 1]
```

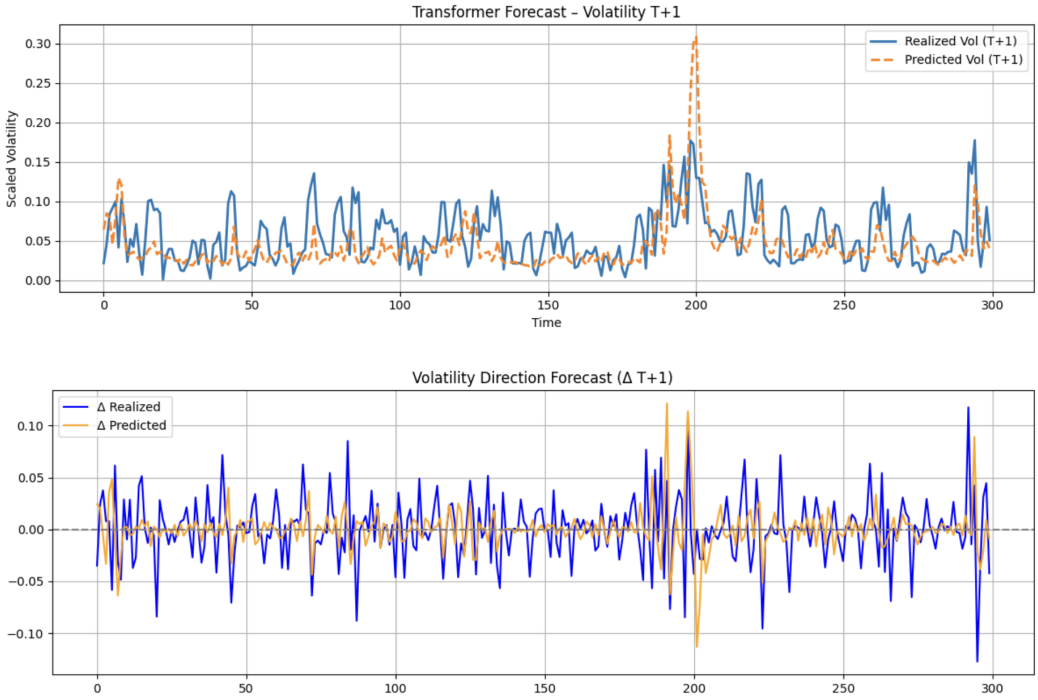


Figure 9: Comparison of Volatility forecast at T+1 and directions vs reality.

Commentary : Sinusoidal Positional Encoding Sinusoidal encodings inject a stable, frequency-aware notion of order that helps the model track periodicities and regime rhythms. In practice, we observe a trade-off: the amplitude is a balanced compromise between the baseline Transformer and the Fourier-enhanced model, while peak detection is clearer and more precise than the baseline. Crucially, it avoids uncontrolled jumps during small, noisy variations, a weakness observed in the Fourier-integrated model. Furthermore, training efficiency is significantly improved, with training time reduced to just above 2 minutes.

The complete notebook, with explanations and code, is available on my GitHub : My GitHub Repository.

References

[1] Bayes, T. (1763). *An Essay towards solving a Problem in the Doctrine of Chances*. Philosophical Transactions of the Royal Society of London, 53, 370–418.

[2] Hardy, G. H., Littlewood, J. E., & Pólya, G. (1934). *Inequalities*. Oxford University Press.

[3] Oppenheim, A. V., & Schafer, R. W. (1989). *Discrete-Time Signal Processing*. Prentice Hall.

[4] Kingma, D. P., & Welling, M. (2013). *Auto-Encoding Variational Bayes*. arXiv preprint arXiv:1312.6114.

[5] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). *Attention is All You Need*. Advances in Neural Information Processing Systems (NeurIPS).

[6] Andersen, T. G., Bollerslev, T., Diebold, F. X., & Labys, P. (2003). *Modeling and Forecasting Realized Volatility*. Econometrica, 71(2), 579–625.